



BUILD



WEB APP
NAVICORE

Build — TraveXperience WebApp

Esta carpeta documenta cómo compilar la aplicación web **TraveXperience**, construida en **React** (frontend) con un **backend en JavaScript** (Node.js). La WebApp incluye dos zonas:

- **Sitio de usuario:** landing, registro/login, descubrimiento, mapa, reservas, pagos, perfil, notificaciones, seguridad, etc.
- **Panel de administrador:** dashboard, inventario de servicios, registro de hoteles/transporte, pagos y configuración de la plataforma.

Nota: aún no se ha confirmado si el frontend usa Vite o Create React App (*react-scripts*). Esta guía cubre ambos casos — usa la sección que corresponda a tu *package.json*.

Requisitos previos

Herramienta	Versión recomendada
Node.js	LTS más reciente (18.x o superior)
npm	Incluido con Node.js
Backend	Node.js con el framework usado (Express u otro)

Instalación de dependencias

Frontend:

```
cd frontend
npm install
```

Backend:

```
cd backend
npm install
```

Variables de entorno

Crear un archivo *.env* en cada proyecto (frontend y backend) con al menos:

Frontend (*.env* — el prefijo depende de la herramienta):

```
# Si usan Vite:
VITE_API_URL=http://localhost:4000

# Si usan Create React App:
REACT_APP_API_URL=http://localhost:4000
```

Backend:

```
PORT=4000
DATABASE_URL=<cadena de conexión de la base de datos>
JWT_SECRET=<clave secreta para tokens de sesión>
```

Ajustar nombres exactos de variables según cómo esté implementado el backend real.

Compilar en desarrollo

Frontend (elige según la herramienta usada):

```
npm run dev      # Vite
# o
npm start        # Create React App
```

Backend:

```
npm run dev      # si usan nodemon u otro watcher
# o
npm start
```

Compilar para producción

Frontend:

```
npm run build
```

- Con **Vite**, esto genera la carpeta *dist/*.
- Con **Create React App**, esto genera la carpeta *build/*.

Para previsualizar el build de producción localmente (solo Vite):

```
npm run preview
```

Backend: normalmente no requiere un paso de “build” si es JS puro (sin TypeScript); simplemente se ejecuta con *npm start* en el servidor de producción.

Estructura relevante del proyecto (según el código revisado)

```
frontend/
├── src/
│   ├── pages/                # Páginas de usuario: Landing, Login, regist
│   │   ├── er,              # recuperarContraseña, Inicio, mapa, infoCa
│   │   ├── rds,            # itinerario, pago, confirmacionPago, favor
│   │   ├── itos,          # notificaciones, perfil, privacidad, segur
│   │   ├── idad,          # historialPagos, 404
│   │   └── pages/admin/   # Páginas de administrador: dashboard, inven
│   └── tario,             # registroHotel, registroTransporte, pagosA
```

```
dmin,
└─ components/      # adminPerfil, configuracionesAdmin,
                    # notificacionesAdmin
                    # Header, Footer, Layout, adminLayout, etc.
```

Notas técnicas encontradas en el código

- El mapa interactivo (*mapa.jsx*) usa **React Leaflet + OpenStreetMap** (no Google Maps), por lo que no requiere ninguna API key.
- Varias páginas de usuario usan un patrón de navegación por *prop onNavigate(pagina)* en lugar de una librería de rutas (*react-router-dom*) — confirmar con el equipo si el proyecto real usa rutas de verdad o este patrón de “SPA de una sola página” con estado.
- El registro (*register.jsx*) permite elegir rol **Usuario** o **Administrador** con reglas de contraseña distintas para cada uno (usuario: 8+ caracteres con letra y número; administrador: 10+ caracteres con mayúscula, minúscula, número y carácter especial).